AI-Assisted Coding
Lab Assignment - 11.1

Name: A. Sushmitha
Roll No: 2303A51541
Batch: 13

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Task Description #1 – Stack Implementation

Task: Use AI to generate a stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

• A functional stack implementation with all required methods and docstrings.

Assignment.py > Stack > pop

```python
class Stack:
    """
    A Stack data structure implementation using a list.

    A Stack follows the Last-In-First-Out (LIFO) principle where the last
    element added is the first one to be removed.
    """
    def __init__(self):
        """
        Initialize an empty stack using a list.
        """
        self.items = []
    def push(self, item):
        """
        Add an item to the top of the stack.

        Args:
            item: The item to be added to the stack.

        Returns:
            None
        """
        self.items.append(item)
    def pop(self):
        """
        Remove and return the item from the top of the stack.
        Returns:
            The item at the top of the stack.
        Raises:
            IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("pop from an empty stack")
```

Assignment.py > ...

```python
class Stack:
    def pop(self):
            raise IndexError("pop from an empty stack")
        return self.items.pop()
    def peek(self):
        """
        Return the item at the top of the stack without removing it.

        Returns:
            The item at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("peek into an empty stack")
        return self.items[-1]
    def is_empty(self):
        """
        Check if the stack is empty.
        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0
if __name__ == "__main__":
    stack = Stack()
    print(f"Is stack empty? {stack.is_empty()}")
    stack.push(10)
    stack.push(20)
    stack.push(30)
    print(f"Is stack empty after pushing items? {stack.is_empty()}")
    print(f"Peek at top: {stack.peek()}")
    print(f"Popped item: {stack.pop()}")  # Output: 30
    print(f"Peek at top after pop: {stack.peek()}")  # Output: 20
    print(f"Popped item: {stack.pop()}")  # Output: 20
    print(f"Popped item: {stack.pop()}")  # Output: 10
    print(f"Is stack empty? {stack.is_empty()}")  # Output: True
```

```
Is stack empty? True
Is stack empty after pushing items? False
Peek at top: 30
Popped item: 30
Peek at top after pop: 20
Popped item: 20
Popped item: 10
Is stack empty? True
```

Explanation:
Implemented a complete stack class in `Assignment.py` with all required methods:

1. `push(item)` – Adds an item to the top of the stack.
2. `pop()` – Removes and returns the top item (raises an error if empty).
3. `peek()` – Returns the top item without removing it (raises an error if empty).
4. `is_empty()` – Checks if the stack is empty.

→ Comprehensive docstrings for the class and each method.
→ Proper error handling with descriptive messages.
→ Example usage demonstrating all methods.
→ The example code follows the LIFO (Last-In-First-Out) principle.

Task Description #2-Queue Implementation Task:

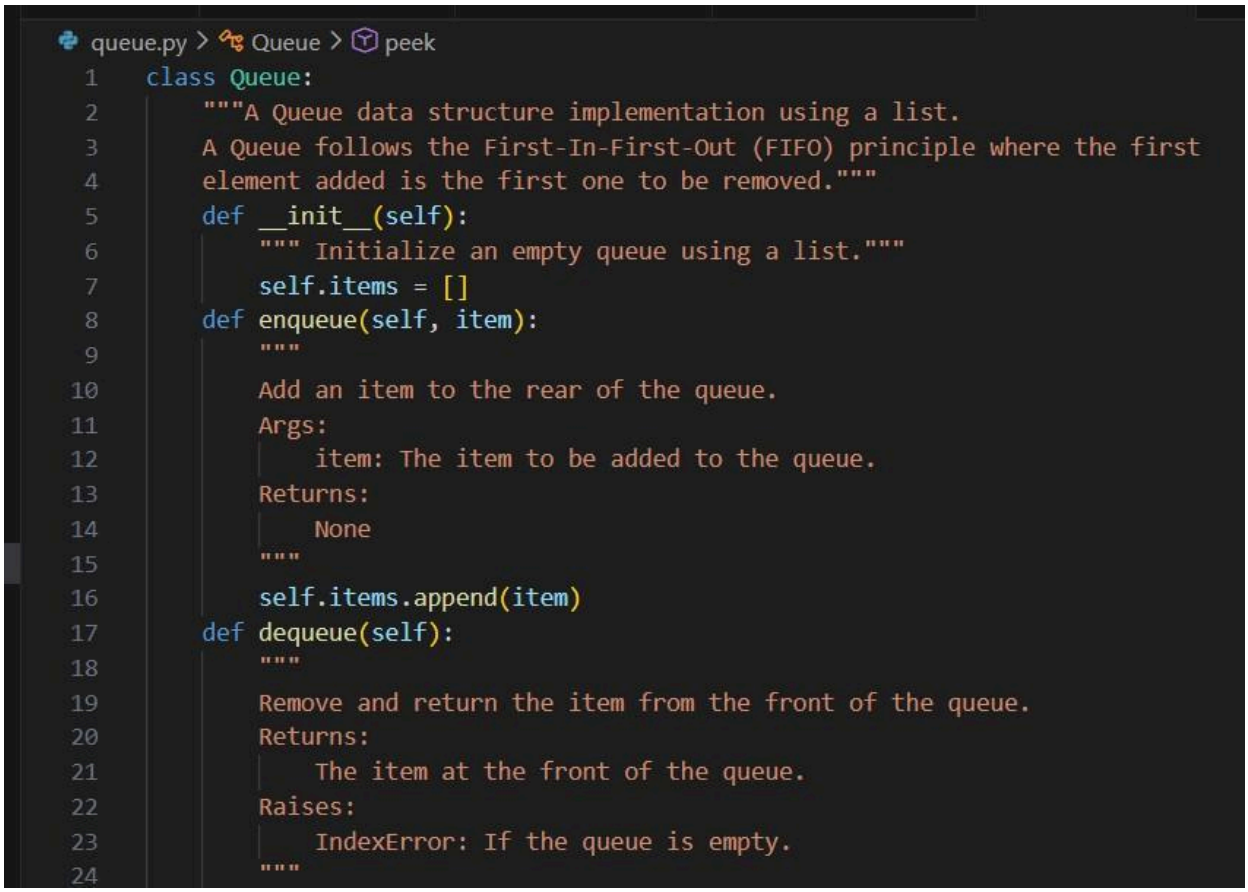
Use AI to implement a queue using Python lists.

Sample Input Code:

class QQueue: pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.



```
        if self.is_empty():
            raise IndexError("dequeue from an empty queue")
        return self.items.pop(0)
    def peek(self):
        """
        Return the item at the front of the queue without removing it.
        Returns:
            The item at the front of the queue.
        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("peek into an empty queue")
```

queue.py > Queue > peek

```
class Queue:
    def peek(self):
        if self.is_empty():
            raise IndexError("peek into an empty queue")
        return self.items[0]
    def size(self):
        """
        Return the number of items in the queue.
        Returns:
            The number of items currently in the queue.
        """
        return len(self.items)
    def is_empty(self):
        """
        Check if the queue is empty.
        Returns:
            True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0
# Example usage and testing
if __name__ == "__main__":
    # Create a new queue
    queue = Queue()
    print("--- Queue Testing ---")
    print(f"Is queue empty? {queue.is_empty()}")  # Output: True
    queue.enqueue(10)
    queue.enqueue(20)
    queue.enqueue(30)
```

```
59      queue.enqueue(10)
60      queue.enqueue(20)
61      queue.enqueue(30)
62      print(f"Is queue empty after enqueuing items? {queue.is_empty()}")  # Output: False
63      print(f"Queue size: {queue.size()}")  # Output: 3
64      print(f"Peek at front: {queue.peek()}")  # Output: 10
65      print(f"Dequeued item: {queue.dequeue()}")  # Output: 10
66      print(f"Peek at front after dequeue: {queue.peek()}")  # Output: 20
67      print(f"Queue size: {queue.size()}")  # Output: 2
68      print(f"Dequeued item: {queue.dequeue()}")  # Output: 20
69      print(f"Dequeued item: {queue.dequeue()}")  # Output: 30
70      print(f"Is queue empty? {queue.is_empty()}")  # Output: True
```

Explanation:

```
--- Queue Testing ---
Is queue empty? True
Is queue empty after enqueuing items? False
Queue size: 3
Peek at front: 10
Dequeued item: 10
Peek at front after dequeue: 20
Queue size: 2
Dequeued item: 20
Dequeued item: 30
Is queue empty? True
```

1. `enqueue(item)` – Adds an item to the rear of the queue.
2. `dequeue()` – Removes and returns the front item (raises an error if empty).
3. `peek()` – Returns the front item without removing it (raises an error if empty).
4. `size()` – Returns the number of items in the queue.
5. `is_empty()` – Checks if the queue is empty.

→ Comprehensive docstrings for the class and each method.
→ Proper error handling with descriptive messages.
→ Queue follows the FIFO (First-In-First-Out) principle.

Task Description #3 – LinkedList

Task: Use AI to generate a singly linked list with insert and display methods.

Sample Input Code:

class Node:

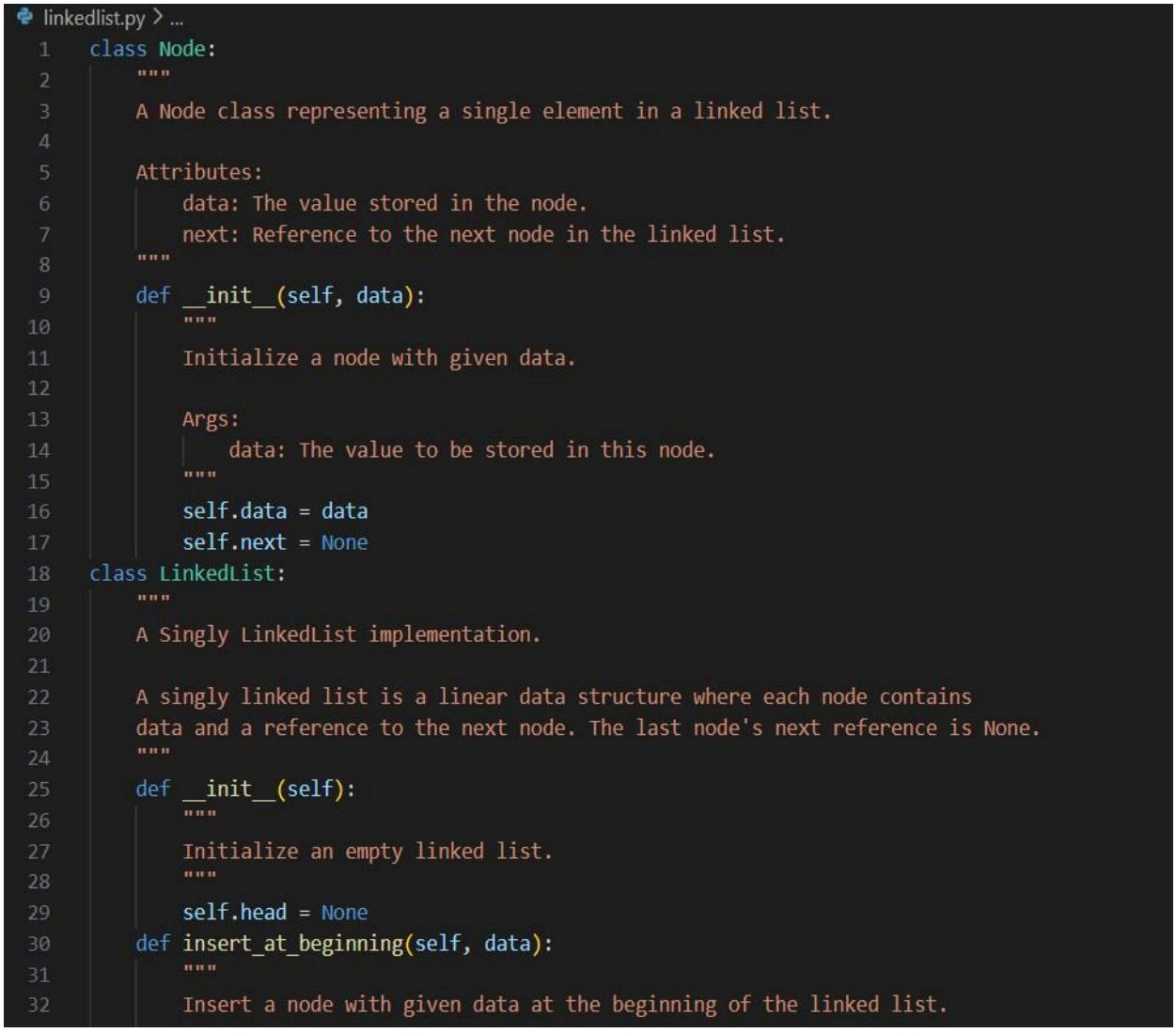
pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.



```python
class Node:
    """
    A Node class representing a single element in a linked list.

    Attributes:
        data: The value stored in the node.
        next: Reference to the next node in the linked list.
    """
    def __init__(self, data):
        """
        Initialize a node with given data.

        Args:
            data: The value to be stored in this node.
        """
        self.data = data
        self.next = None
class LinkedList:
    """
    A Singly LinkedList implementation.

    A singly linked list is a linear data structure where each node contains
    data and a reference to the next node. The last node's next reference is None.
    """
    def __init__(self):
        """
        Initialize an empty linked list.
        """
        self.head = None
    def insert_at_beginning(self, data):
        """
        Insert a node with given data at the beginning of the linked list.
```

linkedlist.py > ...

```
18  class LinkedList:
30      def insert_at_beginning(self, data):
32          Insert a node with given data at the beginning of the linked list.
33          Args:
34              data: The value to be inserted.
35
36          Returns:
37              None
38          """
39          new_node = Node(data)
40          new_node.next = self.head
41          self.head = new_node
42      def insert_at_end(self, data):
43          """
44          Insert a node with given data at the end of the linked list.
45
46          Args:
47              data: The value to be inserted.
48
49          Returns:
50              None
51          """
52          new_node = Node(data)
53          if self.head is None:
54              self.head = new_node
55              return
56          current = self.head
57          while current.next is not None:
58              current = current.next
59          current.next = new_node
60      def insert_at_position(self, data, position):
61          """
```

linkedlist.py > ...

```
18  class LinkedList:
60      def insert_at_position(self, data, position):
62          Insert a node with given data at a specific position in the linked list.
63          Position is 0-indexed. Position 0 inserts at the beginning.
64          If position is greater than the list length, inserts at the end.
65          Args:
66              data: The value to be inserted.
67              position: The position at which to insert (0-indexed).
68          Returns:
69              None
70          """
71          new_node = Node(data)
72          if position == 0:
73              new_node.next = self.head
74              self.head = new_node
75              return
```

```
74              self.head = new_node
75              return
76          current = self.head
77          prev = None
78          count = 0
79          while current is not None and count < position:
80              prev = current
81              current = current.next
82              count += 1
83          if prev is not None:
84              new_node.next = current
85              prev.next = new_node
86      def display(self):
87          """Display all elements in the linked list.
88          Prints the linked list in the format: 1 -> 2 -> 3 -> None
89          Returns:
90              None"""
91          elements = []
```

linkedlist.py > ...

```
18  class LinkedList:
86      def display(self):
91          elements = []
92          current = self.head
93          while current is not None:
94              elements.append(str(current.data))
95              current = current.next
96          print(" -> ".join(elements) + " -> None")
97      def display_as_list(self):
98          """ Return all elements in the linked list as a Python list.
99          Returns:
100             A list containing all elements in the linked list."""
101         elements = []
102         current = self.head
103         while current is not None:
104             elements.append(current.data)
105             current = current.next
106         return elements
107     def is_empty(self):
108         """
109         Check if the linked list is empty.
110         Returns:
111             True if the linked list is empty, False otherwise.
112         """
113         return self.head is None
114     def get_length(self):
115         """
116         Get the length of the linked list.
117         Returns:
118             The number of nodes in the linked list.
119         """
120         count = 0
```

```
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
        return count
if __name__ == "__main__":
    ll = LinkedList()
    print("--- Linked List Testing ---")
    print(f"Is linked list empty? {ll.is_empty()}")  # Output: True
    print("\nInserting 10, 20, 30 at the end:")
    ll.insert_at_end(10)
    ll.insert_at_end(20)
    ll.insert_at_end(30)
    ll.display()  # Output: 10 -> 20 -> 30 -> None
    print("\nInserting 5 at the beginning:")
    ll.insert_at_beginning(5)
    ll.display()  # Output: 5 -> 10 -> 20 -> 30 -> None
    # Insert element at specific position
    print("\nInserting 15 at position 2:")
    ll.insert_at_position(15, 2)
    ll.display()  # Output: 5 -> 10 -> 15 -> 20 -> 30 -> None
    # Check length and if empty
    print(f"\nLinked list length: {ll.get_length()}")  # Output: 5
    print(f"Is linked list empty? {ll.is_empty()}")  # Output: False
    # Display as Python list
    print(f"Linked list as Python list: {ll.display_as_list()}")  # Output: [5, 10, 15, 20, 30]
```

Explanation:

```
--- Linked List Testing ---
Is linked list empty? True

Inserting 10, 20, 30 at the end:
10 -> 20 -> 30 -> None

Inserting 5 at the beginning:
5 -> 10 -> 20 -> 30 -> None

Inserting 15 at position 2:
5 -> 10 -> 15 -> 20 -> 30 -> None

Linked list length: 5
Is linked list empty? False
Linked list as Python list: [5, 10, 15, 20, 30]
```

1. `insert_at_beginning(data)` – Inserts at the start, and `insert_at_end(data)` – Inserts at the end.
2. `insert_at_position(data, position)` – Inserts at the specific position, and `display()` – Prints the linked list.
3. `display_as_list()` – Returns elements as a Python list, and `is_empty()` – Checks if the list is empty.
4. `get_length()` – Returns the number of nodes.

→ All methods include comprehensive docstrings.

Task Description #4-Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

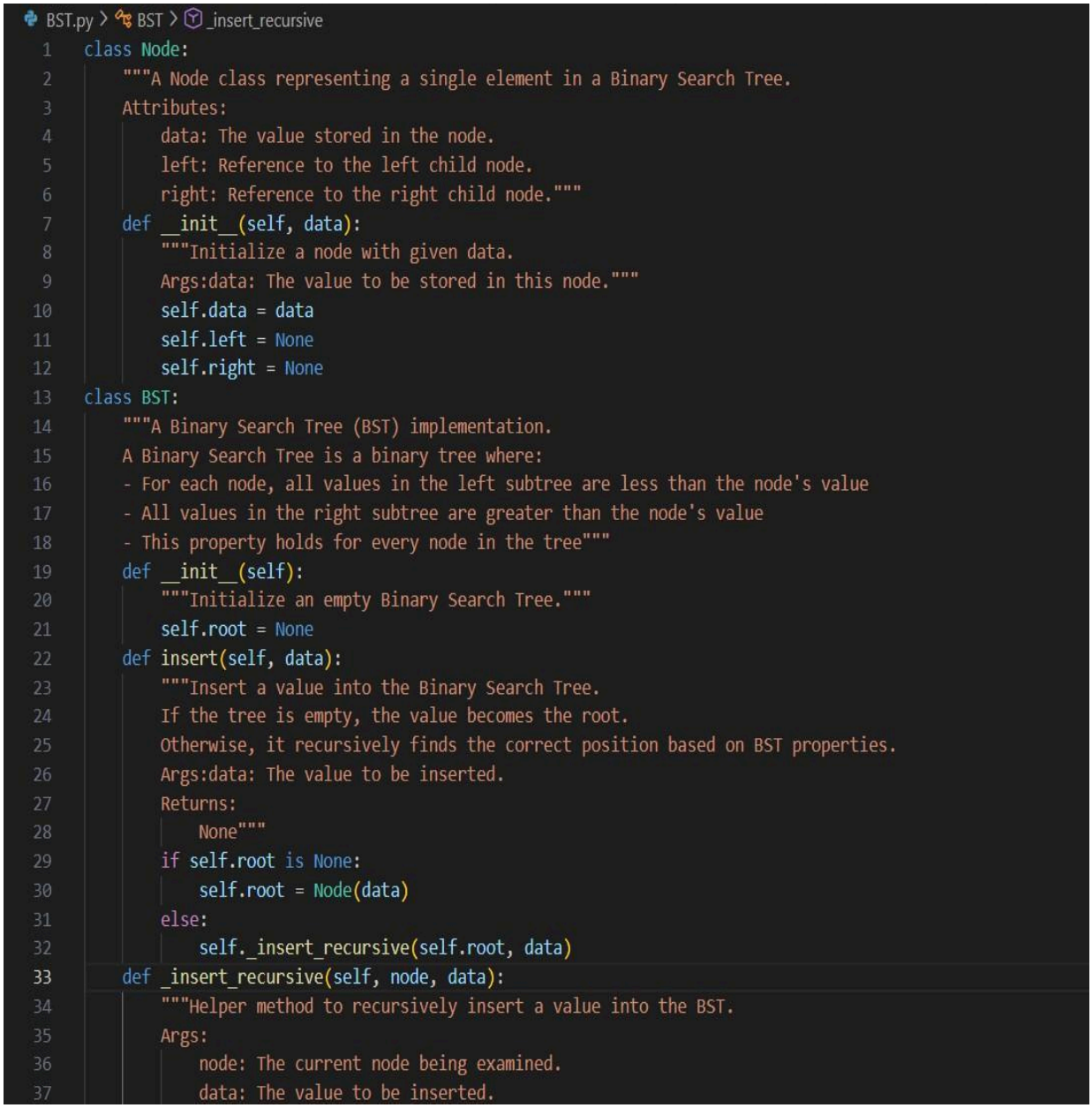
Sample Input Code:

class BST:

pass

Expected Output:

• BST implementation with recursive insert and traversal methods.



BST.py > BST > _insert_recursive

```
class Node:
    """A Node class representing a single element in a Binary Search Tree.
    Attributes:
        data: The value stored in the node.
        left: Reference to the left child node.
        right: Reference to the right child node."""
    def __init__(self, data):
        """Initialize a node with given data.
        Args:data: The value to be stored in this node."""
        self.data = data
        self.left = None
        self.right = None
class BST:
    """A Binary Search Tree (BST) implementation.
    A Binary Search Tree is a binary tree where:
    - For each node, all values in the left subtree are less than the node's value
    - All values in the right subtree are greater than the node's value
    - This property holds for every node in the tree"""
    def __init__(self):
        """Initialize an empty Binary Search Tree."""
        self.root = None
    def insert(self, data):
        """Insert a value into the Binary Search Tree.
        If the tree is empty, the value becomes the root.
        Otherwise, it recursively finds the correct position based on BST properties.
        Args:data: The value to be inserted.
        Returns:
            None"""
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)
    def _insert_recursive(self, node, data):
        """Helper method to recursively insert a value into the BST.
        Args:
            node: The current node being examined.
            data: The value to be inserted.
```

```
            data: The value to be inserted.
        Returns:
            None"""
        if data < node.data:
            if node.left is None:
                node.left = Node(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = Node(data)
            else:
                self._insert_recursive(node.right, data)
    def inorder_traversal(self):
        """Perform in-order traversal of the Binary Search Tree.
        In-order traversal visits nodes in the order: Left -> Node -> Right
        For a BST, this produces values in ascending order.
        Returns:
            A list of values in in-order sequence."""
        result = []
        self._inorder_recursive(self.root, result)
        return result
    def _inorder_recursive(self, node, result):
        """
        Helper method to recursively perform in-order traversal.
        Args:
            node: The current node being visited.
            result: The list to accumulate the traversal results.
```

```
            node: The current node being visited.
            result: The list to accumulate the traversal results.
        Returns:
            None
        """
        if node is not None:
            self._inorder_recursive(node.left, result)
            result.append(node.data)
            self._inorder_recursive(node.right, result)
    def preorder_traversal(self):
        """Perform pre-order traversal of the Binary Search Tree.
        Pre-order traversal visits nodes in the order: Node -> Left -> Right
        Returns:
            A list of values in pre-order sequence."""
        result = []
        self._preorder_recursive(self.root, result)
        return result
    def _preorder_recursive(self, node, result):
        """Helper method to recursively perform pre-order traversal.
        Args:
            node: The current node being visited.
            result: The list to accumulate the traversal results.
        Returns:
            None
        """
        if node is not None:
            result.append(node.data)
            self._preorder_recursive(node.left, result)
            self._preorder_recursive(node.right, result)
    def postorder_traversal(self):
        """Perform post-order traversal of the Binary Search Tree.
        Post-order traversal visits nodes in the order: Left -> Right -> Node
```

```
        Post-order traversal visits nodes in the order: Left -> Right -> Node
        Returns:
            A list of values in post-order sequence."""
        result = []
        self._postorder_recursive(self.root, result)
        return result
    def _postorder_recursive(self, node, result):
        """Helper method to recursively perform post-order traversal.
        Args:
            node: The current node being visited.
            result: The list to accumulate the traversal results.
        Returns:
            None
        """
        if node is not None:
            self._postorder_recursive(node.left, result)
            self._postorder_recursive(node.right, result)
            result.append(node.data)
    def search(self, data):
        """Search for a value in the Binary Search Tree.
        Args:
            data: The value to search for.
        Returns:
            True if the value is found, False otherwise.
        """
        return self._search_recursive(self.root, data)
    def _search_recursive(self, node, data):
        """
        Helper method to recursively search for a value in the BST.
        Args:
            node: The current node being examined.
            data: The value to search for.
```

BST.py > BST > _insert_recursive

```
class BST:
    def _search_recursive(self, node, data):
            data: The value to search for.
        Returns:
            True if the value is found, False otherwise.
        """
        if node is None:
            return False
        if data == node.data:
            return True
        elif data < node.data:
            return self._search_recursive(node.left, data)
        else:
            return self._search_recursive(node.right, data)
    def is_empty(self):
        """Check if the Binary Search Tree is empty.
        Returns:
            True if the tree is empty, False otherwise.
        """
        return self.root is None
    def get_height(self):
        """Get the height of the Binary Search Tree.
        Height is the number of edges in the longest path from root to leaf.
        An empty tree has height -1, and a single node has height 0.
```

```python
    def get_height(self):
        """Get the height of the Binary Search Tree.
        Height is the number of edges in the longest path from root to leaf.
        An empty tree has height -1, and a single node has height 0.
        Returns:
            The height of the tree.
        """
        return self._height_recursive(self.root)
    def _height_recursive(self, node):
        """Helper method to recursively calculate tree height.
        Args:
            node: The current node being examined.
        Returns:
            The height of the subtree rooted at this node.
        """
        if node is None:
            return -1
        return 1 + max(self._height_recursive(node.left),
                       self._height_recursive(node.right))
if __name__ == "__main__":
    bst = BST()
    print("--- Binary Search Tree Testing ---")
    print(f"Is BST empty? {bst.is_empty()}")
    values = [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]
    print(f"\nInserting values: {values}")
    for value in values:
        bst.insert(value)
    print(f"Is BST empty? {bst.is_empty()}")
    print(f"\nIn-order traversal (Ascending): {bst.inorder_traversal()}")
    print(f"Pre-order traversal: {bst.preorder_traversal()}")
    print(f"Post-order traversal: {bst.postorder_traversal()}")
    print(f"\nSearch for 40: {bst.search(40)}")
    print(f"Search for 100: {bst.search(100)}")
    print(f"\nTree height: {bst.get_height()}")
```

```
--- Binary Search Tree Testing ---
Is BST empty? True

Inserting values: [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]
Is BST empty? False

In-order traversal (Ascending): [10, 20, 25, 30, 35, 40, 50, 60, 65, 70, 80]
Pre-order traversal: [50, 30, 20, 10, 25, 40, 35, 70, 60, 65, 80]
Post-order traversal: [10, 25, 20, 35, 40, 30, 65, 60, 80, 70, 50]

Search for 40: True
Search for 100: False

Tree height: 3
```

Explanation:

1. `insert(data)` – Recursively inserts values while maintaining BST properties.
2. `inorder_traversal()` – Left → Node → Right (produces sorted output).
3. `preorder_traversal()` – Node → Left → Right.
4. `postorder_traversal()` – Left → Right → Node.
5. `search(data)` – Searches for a value recursively.
6. `is_empty()` – Checks if the tree is empty.
7. `get_height()` – Returns the height of the tree.

→ All methods use recursive implementations with comprehensive docstrings.

Task Description #5-Hash Table

TTask: UseAI to implement a hash table with basic iinsert, search, anddelete methods.

Sample Input Code:

class HashTable: pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

Hashtable.py > ...

```python
class HashTable:
    """A simple Hash Table implementation using chaining for collision handling.
    - Uses a list of buckets; each bucket is a Python list storing (key, value) pairs.
    - Keys are hashed using Python's built-in `hash()` and the bucket index is
      computed as `hash(key) % capacity`.
    """
    def __init__(self, capacity=16):
        """Initialize the hash table.
        Args:
            capacity (int): Number of buckets to start with (should be > 0)."""
        if capacity <= 0:
            raise ValueError("capacity must be a positive integer")
        self.capacity = capacity
        self.buckets = [[] for _ in range(capacity)]
        self.size = 0
    def _bucket_index(self, key):
        """Return the bucket index for a given key."""
        return hash(key) % self.capacity
    def insert(self, key, value):
        """Insert or update the (key, value) pair into the hash table.
        If the key already exists, its value is updated.
        Collision handling: chaining (append to bucket list).
        """
        idx = self._bucket_index(key)
        bucket = self.buckets[idx]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
```

```
        idx = self._bucket_index(key)
        bucket = self.buckets[idx]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))
        self.size += 1
    def search(self, key):
        """Search for a key and return its value if found, otherwise return None."""
        idx = self._bucket_index(key)
        bucket = self.buckets[idx]
        for k, v in bucket:
            if k == key:
```

Hashtable.py > ...

```
class HashTable:
    def search(self, key):
        bucket = self.buckets[idx]
        for k, v in bucket:
            if k == key:
                return v
        return None
    def delete(self, key):
        """Delete the key from the hash table.
        Returns:
            True if the key was present and deleted, False if the key was not found.
        """
        idx = self._bucket_index(key)
        bucket = self.buckets[idx]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                self.size -= 1
                return True
        return False
    def __len__(self):
        """Return number of elements in the table."""
        return self.size
    def keys(self):
        """Yield all keys stored in the table."""
        for bucket in self.buckets:
            for k, _ in bucket:
                yield k
    def values(self):
        """Yield all values stored in the table."""
        for bucket in self.buckets:
            for _, v in bucket:
                yield v
    def items(self):
        """Yield all (key, value) pairs stored in the table."""
        for bucket in self.buckets:
            for kv in bucket:
```

```
    def items(self):
        """Yield all (key, value) pairs stored in the table."""
        for bucket in self.buckets:
            for kv in bucket:
                yield kv
if __name__ == "__main__":
    ht = HashTable(capacity=8)
    ht.insert("apple", 3)
    ht.insert("banana", 5)
    ht.insert("orange", 2)
    ht.insert("apple", 10)  # update
    print("Keys:", list(ht.keys()))
    print("Values:", list(ht.values()))
    print("Items:", list(ht.items()))
    print("Search apple:", ht.search("apple"))  # 10
    print("Search grape:", ht.search("grape"))  # None
    print("Delete banana:", ht.delete("banana"))  # True
    print("Delete grape:", ht.delete("grape"))    # False
    print("Items after deletion:", list(ht.items()))
    print("Length:", len(ht))
```

Explanation:

```
Keys: ['apple', 'banana', 'orange']
Values: [10, 5, 2]
Items: [('apple', 10), ('banana', 5), ('orange', 2)]
Search apple: 10
Search grape: None
Delete banana: True
Delete grape: False
Items after deletion: [('apple', 10), ('orange', 2)]
Length: 2
```

1. `insert(key, value)` – Adds or updates a (key, value) pair. If the key exists in its bucket, the value is replaced; otherwise, the pair is appended and `self.size` increments. Uses chaining for collisions.
2. `search(key)` – Returns the value for the key if present; otherwise, returns `None`.
3. `delete(key)` – Removes the key if present, decrements `self.size`, and returns `True`; returns `False` if the key wasn't found.
4. `search(key)` – Returns the value for the key if present; otherwise, returns `None`.
5. `len()` – Returns the number of stored elements (`self.size`).

→ `keys()`, `values()`, `items()` – Generators yielding stored keys, values, and (key, value) pairs, respectively.

Task Description #6-Graph Representation

Task: use AI to implement a graph using an adjacency list.

Sample Input Code:

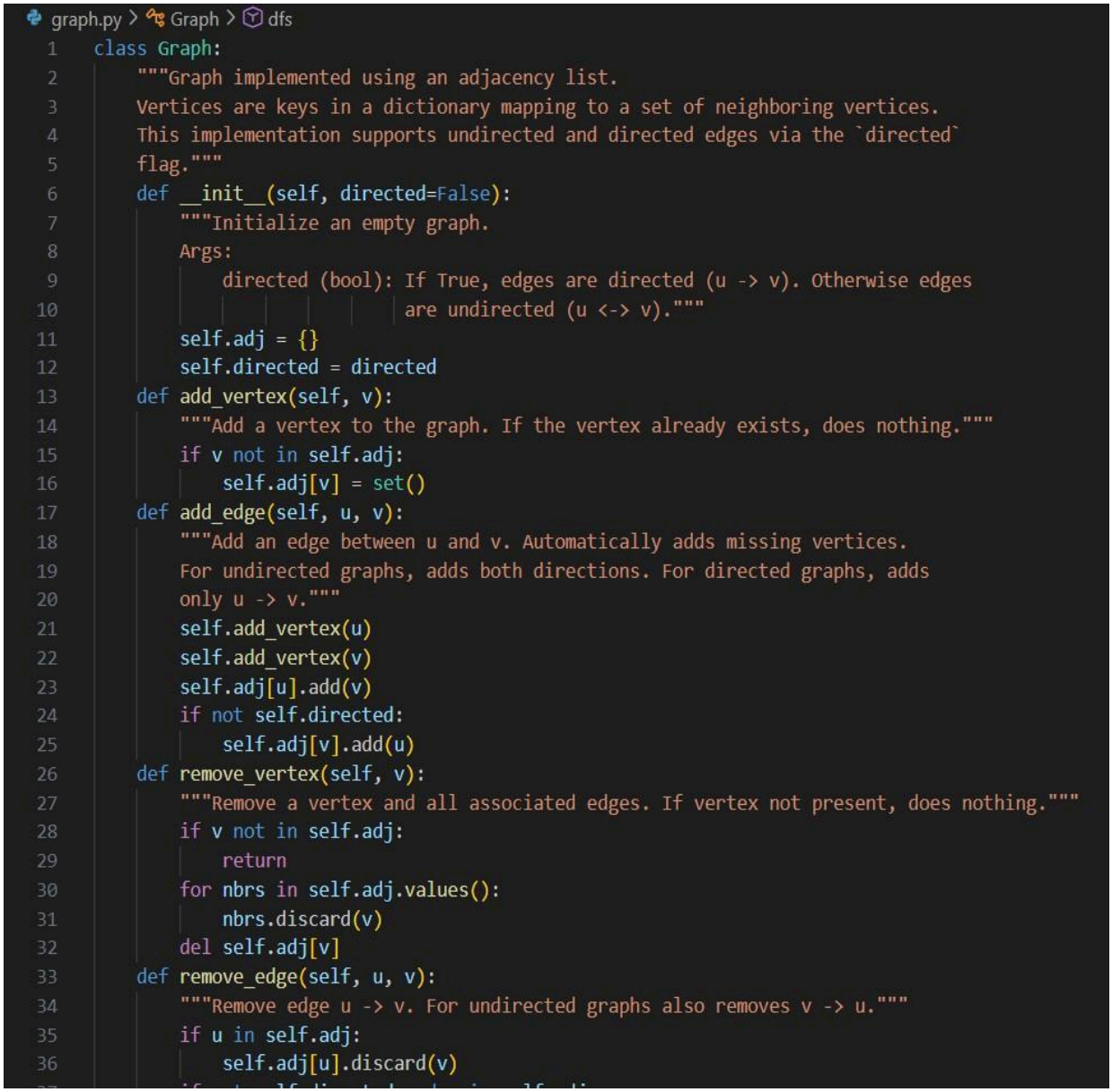
class Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges,

and display connections.



graph.py > Graph > dfs

```python
class Graph:
    """Graph implemented using an adjacency list.
    Vertices are keys in a dictionary mapping to a set of neighboring vertices.
    This implementation supports undirected and directed edges via the `directed`
    flag."""
    def __init__(self, directed=False):
        """Initialize an empty graph.
        Args:
            directed (bool): If True, edges are directed (u -> v). Otherwise edges
                             are undirected (u <-> v)."""
        self.adj = {}
        self.directed = directed
    def add_vertex(self, v):
        """Add a vertex to the graph. If the vertex already exists, does nothing."""
        if v not in self.adj:
            self.adj[v] = set()
    def add_edge(self, u, v):
        """Add an edge between u and v. Automatically adds missing vertices.
        For undirected graphs, adds both directions. For directed graphs, adds
        only u -> v."""
        self.add_vertex(u)
        self.add_vertex(v)
        self.adj[u].add(v)
        if not self.directed:
            self.adj[v].add(u)
    def remove_vertex(self, v):
        """Remove a vertex and all associated edges. If vertex not present, does nothing."""
        if v not in self.adj:
            return
        for nbrs in self.adj.values():
            nbrs.discard(v)
        del self.adj[v]
    def remove_edge(self, u, v):
        """Remove edge u -> v. For undirected graphs also removes v -> u."""
        if u in self.adj:
            self.adj[u].discard(v)
```

```
        if u in self.adj:
            self.adj[u].discard(v)
        if not self.directed and v in self.adj:
            self.adj[v].discard(u)
    def neighbors(self, v):
        """
        Return an iterable of neighbors for vertex v. Raises KeyError if v missing.
        """
        return iter(self.adj[v])
    def has_vertex(self, v):
        """Return True if vertex exists in the graph."""
        return v in self.adj
    def has_edge(self, u, v):
        """Return True if edge u -> v exists."""
        return u in self.adj and v in self.adj[u]
    def display(self):
        """Print adjacency list of the graph."""
        for v in sorted(self.adj):
            nbrs = ', '.join(map(str, sorted(self.adj[v])))
            print(f"{v}: {nbrs}")
    def bfs(self, start):
        """
        Breadth-first search from start vertex. Returns list of visited vertices
        in BFS order. Raises KeyError if start not in graph.
        """
        if start not in self.adj:
            raise KeyError(f"Vertex {start} not in graph")
        visited = set()
        order = []
        from collections import deque
        q = deque([start])
        visited.add(start)
        while q:
            u = q.popleft()
            order.append(u)
```

```
            order.append(u)
            for v in self.adj[u]:
                if v not in visited:
                    visited.add(v)
                    q.append(v)
        return order
    def dfs(self, start):
        """Depth-first search (iterative) from start vertex. Returns list of visited
        vertices in DFS order. Raises KeyError if start not in graph."""
        if start not in self.adj:
            raise KeyError(f"Vertex {start} not in graph")
        visited = set()
        order = []
        stack = [start]
        while stack:
            u = stack.pop()
            if u in visited:
                continue
            visited.add(u)
            order.append(u)
```

```
                continue
            visited.add(u)
            order.append(u)
            for v in sorted(self.adj[u], reverse=True):
                if v not in visited:
                    stack.append(v)
        return order
if __name__ == "__main__":
    g = Graph()
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.add_edge('C', 'E')
    print("Adjacency list:")
    g.display()
    print("\nBFS from A:", g.bfs('A'))
    print("DFS from A:", g.dfs('A'))
```

```
Adjacency list:
A: B, C
B: A, D
C: A, E
D: B
E: C

BFS from A: ['A', 'C', 'B', 'E', 'D']
DFS from A: ['A', 'B', 'D', 'C', 'E']
```

Explanation:

1. Implements a graph using an adjacency list (supports directed or undirected graphs). `self.adj` is a dictionary mapping each vertex → set of neighboring vertices.

2. `add_edge` auto-creates missing vertices; undirected graphs add both directions; `bfs`/`dfs` return visitation order lists.

Complexity: Adding/removing an edge or checking membership is O(1) on average; traversals (`bfs`/`dfs`) are O(V + E).

Create: `g = Graph()`

Adding: `g.add_edge('A', 'B')`

Showing: `g.display()`

Traverse: `g.bfs('A')`, `g.dfs('A')`

Task Description #7-Priority Queue

Task: Use AI to implement a priority queue using python's heapq module

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```python
import heapq
class PriorityQueue:
    """A Priority Queue implementation using Python's heapq module.
    Items are dequeued based on priority. By default, higher priority values
    are dequeued first (max-heap behavior). Uses a min-heap internally with
    negated priorities."""
    def __init__(self):
        """Initialize an empty priority queue."""
        self.heap = []
        self.counter = 0  # Tie-breaker for stable ordering when priorities are equal
    def enqueue(self, item, priority):
        """Add an item to the queue with a given priority.
        Higher priority values are dequeued first. If two items have the same
        priority, they are dequeued in the order they were added.
        Args:
            item: The data to be stored.
            priority (int/float): The priority value. Higher = higher priority.
        Returns:
            None """
        heapq.heappush(self.heap, (-priority, self.counter, item))
        self.counter += 1
    def dequeue(self):
        """Remove and return the item with the highest priority.
        Returns:
            The item with the highest priority value.
        Raises:
            IndexError: If the queue is empty."""
        if self.is_empty():
            raise IndexError("dequeue from an empty priority queue")
        _, _, item = heapq.heappop(self.heap)
        return item
    def peek(self):
        """Return the item with the highest priority without removing it.
        Returns:
            The item with the highest priority value.
        Raises:
            IndexError: If the queue is empty.
```

```
            The item with the highest priority value.
        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("peek into an empty priority queue")
        _, _, item = self.heap[0]
        return item
    def is_empty(self):
        """Check if the priority queue is empty.
        Returns:
            True if the queue is empty, False otherwise.
        """
        return len(self.heap) == 0
    def size(self):
        """Return the number of items in the priority queue.
        Returns:
            The number of items in the queue."""
        return len(self.heap)
    def display(self):
        """Display all items in the priority queue in priority order (highest first).
        Prints items along with their actual (non-negated) priority values. """
        if self.is_empty():
            print("Priority Queue is empty")
            return
        items_with_priority = [(-priority, item) for priority, _, item in self.heap]
        items_with_priority.sort(reverse=True)
        print("Priority Queue (highest to lowest priority):")
        for priority, item in items_with_priority:
            print(f"  Item: {item}, Priority: {priority}")
if __name__ == "__main__":
    pq = PriorityQueue()
    print("--- Priority Queue Testing ---")
    print(f"Is priority queue empty? {pq.is_empty()}")  # True
    print("\nEnqueueing items:")
```

```
            print(f"  Item: {item}, Priority: {priority}")
if __name__ == "__main__":
    pq = PriorityQueue()
    print("--- Priority Queue Testing ---")
    print(f"Is priority queue empty? {pq.is_empty()}")  # True
    print("\nEnqueueing items:")
    pq.enqueue("Task A", 3)
    pq.enqueue("Task B", 5)
    pq.enqueue("Task C", 1)
    pq.enqueue("Task D", 5)
    pq.enqueue("Task E", 2)
    pq.display()
    print(f"\nQueue size: {pq.size()}")  # 5
    print(f"Peek at highest priority: {pq.peek()}")  # Task B or Task D (priority 5)
    print("\nDequeuing items in priority order:")
    while not pq.is_empty():
        item = pq.dequeue()
        print(f"  Dequeued: {item}")

    print(f"\nIs priority queue empty? {pq.is_empty()}")  # True

```

Explanation:

1. `enqueue(item, priority)` – Adds an item with a priority (higher values = higher priority).
2. `dequeue()` – Removes and returns the highest-priority item.
3. `peek()` – Returns the highest-priority item without removing it.
4. `display()` – Shows all items sorted by priority (highest first).
5. `is_empty()`, `size()` – Utility methods.

Task Description #8 – Deque

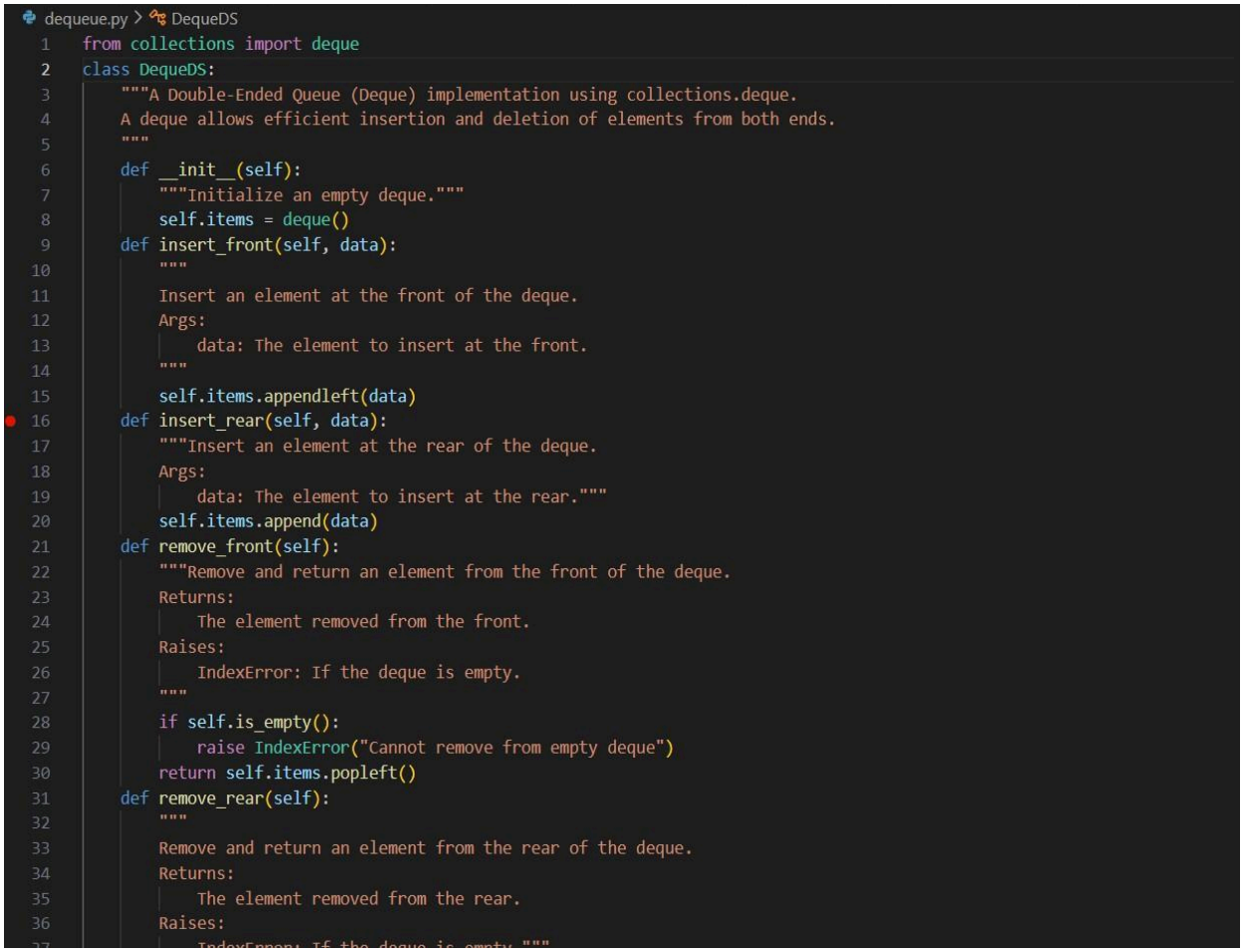
Task: Use AI to implement a double-ended queue using collections.
deque.

Sample Input Code:

class DequeDS:
pass

Expected Output:

• Insert and remove from both ends with docstrings.



```
        Raises:
            IndexError: If the deque is empty."""
        if self.is_empty():
            raise IndexError("Cannot remove from empty deque")
        return self.items.pop()
    def is_empty(self):
        """
        Check if the deque is empty.
        Returns:
            True if the deque is empty, False otherwise.
        """
        return len(self.items) == 0
    def size(self):
        """
        Get the number of elements in the deque.
        Returns:
            The number of elements in the deque."""
        return len(self.items)
    def display(self):
        """Display the elements of the deque.
        Returns:
            A string representation of the deque."""
        return str(list(self.items))
if __name__ == "__main__":
    dq = DequeDS()
    print("Inserting elements...")
    dq.insert_rear(10)
    dq.insert_rear(20)
    dq.insert_front(5)
    dq.insert_rear(30)
    print(f"Deque after insertions: {dq.display()}")
    print(f"Size: {dq.size()}")
```

```
    dq.insert_rear(30)
    print(f"Deque after insertions: {dq.display()}")
    print(f"Size: {dq.size()}")
    print("\nRemoving elements...")
    front_element = dq.remove_front()
    print(f"Removed from front: {front_element}")
    rear_element = dq.remove_rear()
    print(f"Removed from rear: {rear_element}")
    print(f"Deque after removals: {dq.display()}")
    print(f"Size: {dq.size()}")
    print(f"Is empty: {dq.is_empty()}")
```

```
Inserting elements...
Deque after insertions: [5, 10, 20, 30]
Size: 4

Removing elements...
Removed from front: 5
Removed from rear: 30
Deque after removals: [10, 20]
Size: 2
Is empty: False
```

Explanation:

1. `insert_front(data)` – Insert element at the front (left end).
2. `insert_rear(data)` – Insert element at the rear (right end).
3. `remove_front()` – Remove element from the front.
4. `remove_rear()` – Remove element from the rear.
5. `is_empty()` – Check if the deque is empty.
6. `size()` – Get the number of elements.
7. `display()` – Display all elements.

→ All methods include comprehensive docstrings explaining their purpose, parameters, return values, and exceptions.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure
Scenario: Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph o Hash Table
o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

application.py > ...

```python
class EventRegistration:
    """Simple Event Registration System using Hash Table."""
    def __init__(self):
        """Initialize with empty events dictionary."""
        self.events = {}
    def create_event(self, event_id, event_name, capacity):
        """Create a new event."""
        self.events[event_id] = {
            'name': event_name,
            'capacity': capacity,
            'participants': {}
        }
        print(f"✓ Event '{event_name}' created (Capacity: {capacity})")
    def register(self, event_id, participant_id, name):
        """Register a participant. Returns True if successful."""
        if event_id not in self.events:
            print(f"✗ Event not found")
            return False
        event = self.events[event_id]
        if len(event['participants']) >= event['capacity']:
            print(f"✗ Event full")
            return False
        if participant_id in event['participants']:
            print(f"✗ Already registered")
            return False
        event['participants'][participant_id] = name
        print(f"✓ {name} registered for '{event['name']}'")
        return True
    def remove(self, event_id, participant_id):
        """Remove a participant."""
        if event_id in self.events and participant_id in self.events[event_id]['participants']:
            name = self.events[event_id]['participants'].pop(participant_id)
            print(f"✓ {name} removed")
            return True
        print(f"✗ Participant not found")
        return False
    def show_participants(self, event_id):
        """Show all participants in an event."""
        if event_id not in self.events:
            print(f"✗ Event not found")
            return
        event = self.events[event_id]
        count = len(event['participants'])
        capacity = event['capacity']
        print(f"\n{event['name']}: {count}/{capacity} participants")
        for pid, name in event['participants'].items():
            print(f"  - {pid}: {name}")
if __name__ == "__main__":
```

```python
if __name__ == "__main__":
    reg = EventRegistration()
    reg.create_event("E001", "Tech Summit", 3)
    reg.create_event("E002", "AI Workshop", 2)
    print("\n--- Registrations ---")
    reg.register("E001", "P1", "Alice")
    reg.register("E001", "P2", "Bob")
    reg.register("E001", "P3", "Charlie")
    reg.register("E001", "P4", "Diana")  # Over capacity
    reg.show_participants("E001")
    print("\n--- Removal ---")
    reg.remove("E001", "P2")
    reg.show_participants("E001")
```

Explanation:

```
✓ Event 'Tech Summit' created (Capacity: 3)
✓ Event 'AI Workshop' created (Capacity: 2)

--- Registrations ---
✓ Alice registered for 'Tech Summit'
✓ Bob registered for 'Tech Summit'
✓ Charlie registered for 'Tech Summit'
✗ Event full

Tech Summit: 3/3 participants
  - P1: Alice
  - P2: Bob
  - P3: Charlie

--- Removal ---
✓ Bob removed

Tech Summit: 2/3 participants
  - P1: Alice
  - P3: Charlie
```

1. `create_event()` – Create events with capacity.
2. `register()` – Register participants (with capacity and duplicate checks).
3. `remove()` – Remove participants.
4. `remove()` – Remove participants.
5. `show_participants()` – Display all registered participants.

Feature	Data Structure	Justification
Student Attendance Tracking	Hash Table	O(1) lookup to check student status (in/out) and quickly update their presence records
Event Registration System	Hash Table	O(1) search and removal of participants; ideal for instant registration confirmation and easy management
Library Book Borrowing	Priority Queue	Efficiently tracks and prioritizes overdue books; O(1) access to most urgent items
Bus Scheduling System	Graph	Models stops as vertices and routes as edges; perfect for pathfinding and visualizing connections
Cafeteria Order Queue	Queue (Deque)	Implements FIFO service; students served in arrival order with O(1) enqueue/dequeue

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST)

o Graph o Hash Tableo Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

ecommerce.py > ...

```python
from collections import deque
class ShoppingCart:
    """Shopping cart using Deque for dynamic add/remove."""
    def __init__(self):
        """Initialize empty cart."""
        self.items = deque()
    def add_product(self, product_id, product_name, price, quantity=1):
        """Add product to cart."""
        self.items.append({'id': product_id, 'name': product_name, 'price': price, 'qty': quantity})
        print(f"✓ Added {quantity} x {product_name} (${price})")
    def remove_product(self, product_id):
        """Remove product from cart."""
        for item in self.items:
            if item['id'] == product_id:
                self.items.remove(item)
                print(f"✓ Removed {item['name']}")
                return True
        print(f"X Product not found")
        return False
    def get_total(self):
        """Calculate total cart value."""
        return sum(item['price'] * item['qty'] for item in self.items)
    def display_cart(self):
        """Show cart contents."""
        if not self.items:
            print("Cart is empty")
            return
        print("\n--- SHOPPING CART ---")
        total = 0
        for item in self.items:
            subtotal = item['price'] * item['qty']
            print(f"{item['name']} x{item['qty']}: ${subtotal:.2f}")
            total += subtotal
        print(f"Total: ${total:.2f}\n")
if __name__ == "__main__":
    cart = ShoppingCart()
```

```python
if __name__ == "__main__":
    cart = ShoppingCart()
    print("--- Adding Products ---")
    cart.add_product(101, "Laptop", 999.99, 1)
    cart.add_product(102, "Mouse", 29.99, 2)
    cart.add_product(103, "Keyboard", 79.99, 1)
    cart.display_cart()
    print("--- Removing Product ---")
    cart.remove_product(102)
    cart.display_cart()
    print(f"Final Total: ${cart.get_total():.2f}")

```

Explanation:

Shopping Cart → Deque (add/remove both ends)
Order Processing → Queue (FIFO order)
Top-Selling Products → Priority Queue (rank by sales)
Product Search → Hash Table (O(1) lookup)
Delivery Routes → Graph (warehouse connections)

1. `add_product()` – Add items with quantity.
2. `remove_product()` – Remove by product ID.
3. `get_total()` – Calculate cart value.
4. `display_cart()` – Show formatted cart

Feature	Data Structure	Justification
Shopping Cart Management	Deque	Add/remove products easily from both ends
Order Processing System	Queue	FIFO – process orders in arrival order
Top-Selling Products Tracker	Priority Queue	Rank products by sales count
Product Search Engine	Hash Table	O(1) fast lookup using product ID
Delivery Route Planning	Graph	Connect warehouses and model delivery routes